



ScholarNest

Spark Azure Databricks

Databricks Spark Certification and beyond

Instructor: Prashant Kumar Pandey



Azure
Databricks

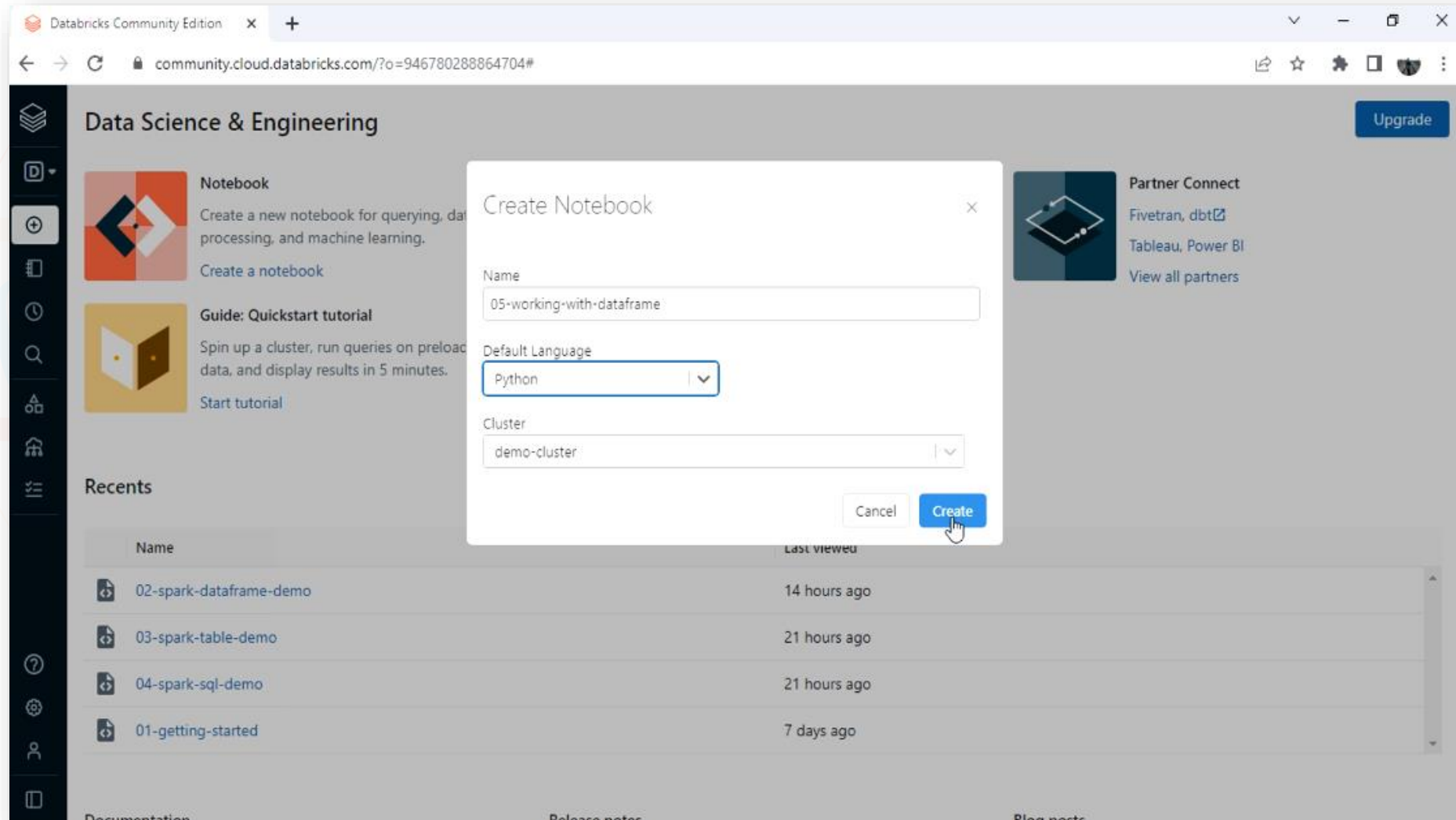


Absolute Beginner to Specialization in Apache Spark and Azure Databricks



Applying Dataframe Transformations

Go to your Databricks workspace and create a new notebook as given below. **(Reference: 05-working-with-dataframe.ipynb)**



Here we use the older code which we used earlier to create a data frame for the fire call response data set.



The screenshot shows a Databricks workspace interface. At the top, the tab is labeled "05-working-with-dataframe" with a "Python" language selector. Below the tab, the cluster name "demo-cluster" is visible. The command editor shows a Python script for reading a CSV file. The script is as follows:

```
1 raw_fire_df = spark.read \  
2     .format("csv") \  
3     .option("header", "true") \  
4     .option("inferSchema", "true") \  
5     .load("/databricks-datasets/learning-spark-v2/sf-fire/sf-fire-calls.csv")
```

Below the code, it indicates "(2) Spark Jobs". At the bottom, a status message reads: "Command took 1.28 minutes -- by prashant@scholarnest.com at 4/7/2022, 11:30:34 AM on demo-cluster".

If we display the older raw_fire_df Dataframe and see the results as given below. We can see there are two problems in this Dataframe:

1. Column Names are not standardized
2. Date fields are of string type

05-working-with-dataframe Python

demo-cluster

Command took 1.28 minutes -- by prashant@scholarnest.com at 4/7/2022, 11:30:34 AM on demo-cluster

Cmd 2

```
1 display(raw_fire_df)
```

(1) Spark Jobs

	Call Number	Unit ID	Incident Number	CallType	Call Date	Watch Date	Call Final Disposition	Available DtTm	Add
1	20110014	M29	2003234	Medical Incident	01/11/2002	01/10/2002	Other	01/11/2002 01:58:43 AM	10TH
2	20110015	M08	2003233	Medical Incident	01/11/2002	01/10/2002	Other	01/11/2002 02:10:17 AM	300
3	20110016	B02	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:47:00 AM	200C
4	20110016	B04	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:51:54 AM	200C
5	20110016	D2	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:47:00 AM	200C
6	20110016	E03	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:47:00 AM	200C
7	20110016	E38	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:51:17 AM	200C

Truncated results, showing first 1000 rows.
[Click to re-execute with maximum result limits.](#)

Command took 1.83 seconds -- by prashant@scholarnest.com at 4/7/2022, 11:32:44 AM on demo-cluster

Column Names are not standardized:

You can see a blank space between call and number. That's not standard. Spark Dataframe column names are case insensitive. However, space in a column name is not a common thing. Let me fix these column names and rename them to remove spaces.

05-working-with-dataframe Python

demo-cluster

Command took 1.28 minutes -- by prashant@scholarnest.com at 4/7/2022, 11:30:34 AM on demo-cluster

Cmd 2

```
1 display(raw_fire_df)
```

(1) Spark Jobs

	Call Number	Unit ID	Incident Number	CallType	Call Date	Watch Date	Call Final Disposition	Available DtTm	Add
1	20110014	M29	2003234	Medical Incident	01/11/2002	01/10/2002	Other	01/11/2002 01:58:43 AM	10TH
2	20110015	M08	2003233	Medical Incident	01/11/2002	01/10/2002	Other	01/11/2002 02:10:17 AM	300
3	20110016	B02	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:47:00 AM	200C
4	20110016	B04	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:51:54 AM	200C
5	20110016	D2	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:47:00 AM	200C
6	20110016	E03	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:47:00 AM	200C
7	20110016	E38	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:51:17 AM	200C

Truncated results, showing first 1000 rows.
[Click to re-execute with maximum result limits.](#)

Command took 1.83 seconds -- by prashant@scholarnest.com at 4/7/2022, 11:32:44 AM on demo-cluster

Cmd 3

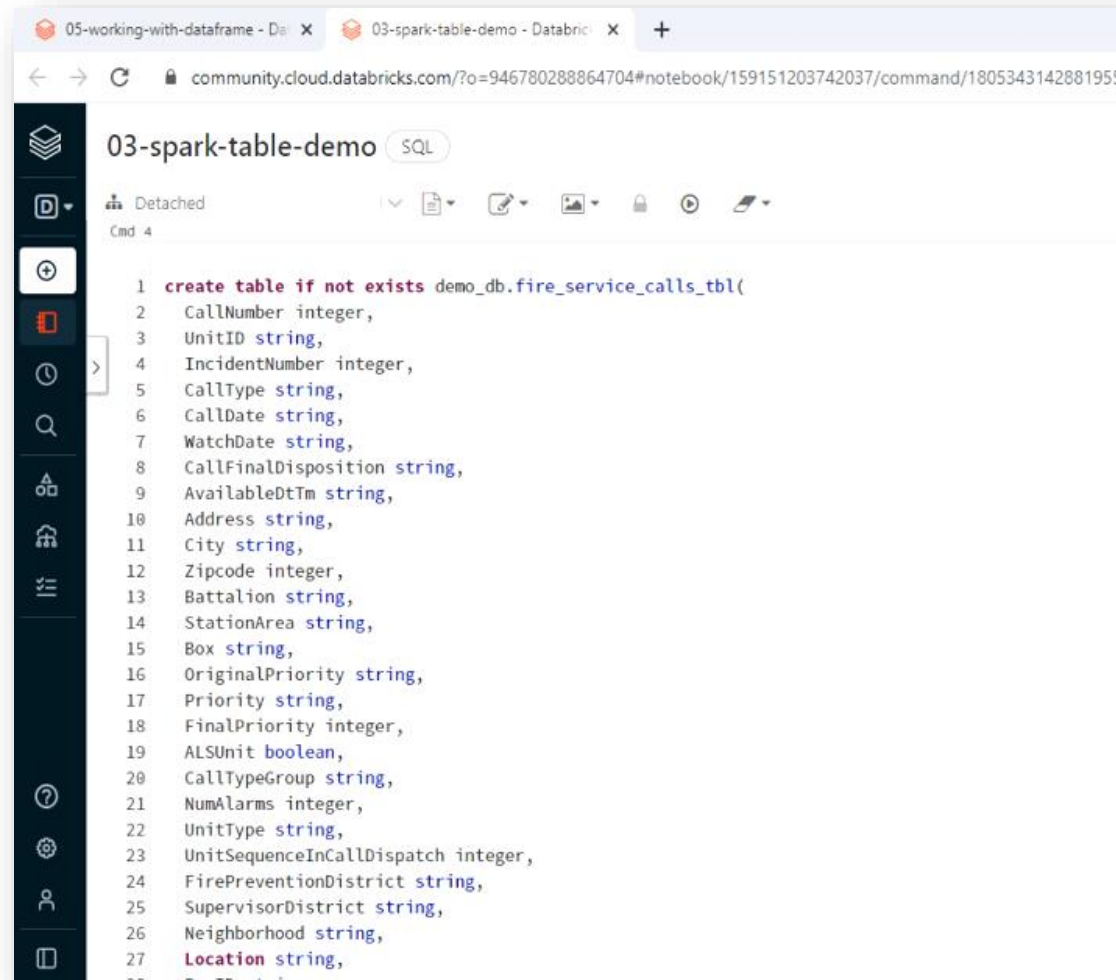
1. Column Names are not standardized

2. Date fields are of string type

Column Names are not standardized:

We fixed this problem while creating the table. (Reference: 03-spark-table-demo.sql)

You can see in the image below that we created column names without spaces when we created the table. Then we loaded data into this table. So we already fixed the column names while creating a table.



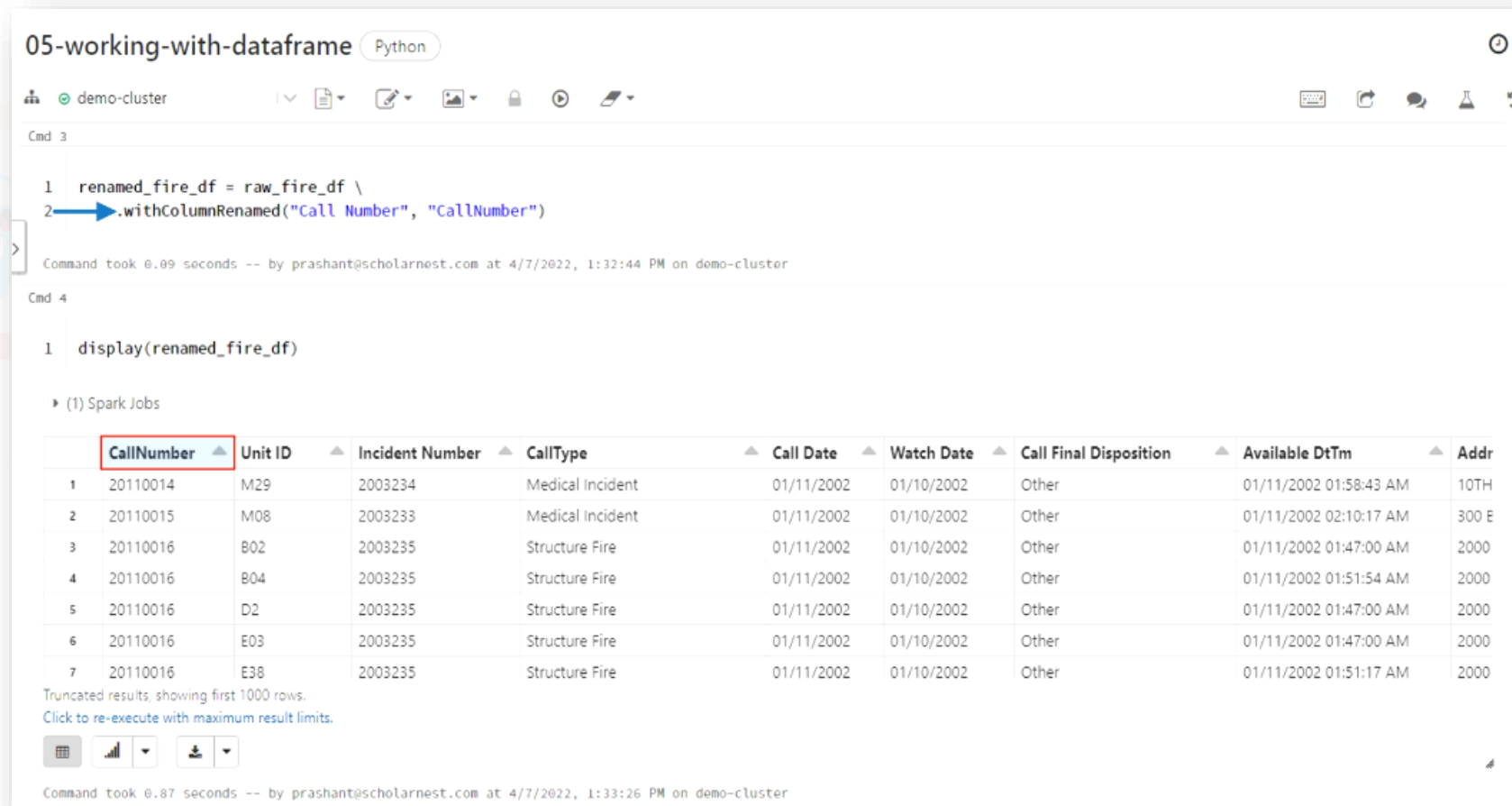
The screenshot shows a Databricks notebook interface. The title bar indicates the notebook is named '03-spark-table-demo'. The main area displays a SQL command in a code editor, labeled 'Cmd 4'. The command is a 'CREATE TABLE' statement for a table named 'demo_db.fire_service_calls_tbl'. The columns listed are: CallNumber (integer), UnitID (string), IncidentNumber (integer), CallType (string), CallDate (string), WatchDate (string), CallFinalDisposition (string), AvailableDtTm (string), Address (string), City (string), Zipcode (integer), Battalion (string), StationArea (string), Box (string), OriginalPriority (string), Priority (string), FinalPriority (integer), ALSUnit (boolean), CallTypeGroup (string), NumAlarms (integer), UnitType (string), UnitSequenceInCallDispatch (integer), FirePreventionDistrict (string), SupervisorDistrict (string), Neighborhood (string), and Location (string). The column names are all in lowercase and contain no spaces, demonstrating standardized naming.

```
1 create table if not exists demo_db.fire_service_calls_tbl(  
2   CallNumber integer,  
3   UnitID string,  
4   IncidentNumber integer,  
5   CallType string,  
6   CallDate string,  
7   WatchDate string,  
8   CallFinalDisposition string,  
9   AvailableDtTm string,  
10  Address string,  
11  City string,  
12  Zipcode integer,  
13  Battalion string,  
14  StationArea string,  
15  Box string,  
16  OriginalPriority string,  
17  Priority string,  
18  FinalPriority integer,  
19  ALSUnit boolean,  
20  CallTypeGroup string,  
21  NumAlarms integer,  
22  UnitType string,  
23  UnitSequenceInCallDispatch integer,  
24  FirePreventionDistrict string,  
25  SupervisorDistrict string,  
26  Neighborhood string,  
27  Location string,  
28  ...)
```

Column Names are not standardized:

Here is how we can fix the column names while working on Dataframes. **(Reference: 05-working-with-dataframe.ipynb)**

Spark Dataframe offers you withColumnRenamed() transformation to rename a column. You can see in the screenshot below that withColumnRenamed() takes two arguments: Old column name and New column name. And we can see the desired results when we are displaying the Dataframe below.



The screenshot shows a Jupyter Notebook interface with the title "05-working-with-dataframe" and a "Python" language selector. The notebook is running on a "demo-cluster".

Cmd 3:

```
1 renamed_fire_df = raw_fire_df \
2   .withColumnRenamed("Call Number", "CallNumber")
```

Command took 0.09 seconds -- by prashant@scholarnest.com at 4/7/2022, 1:32:44 PM on demo-cluster

Cmd 4:

```
1 display(renamed_fire_df)
```

▶ (1) Spark Jobs

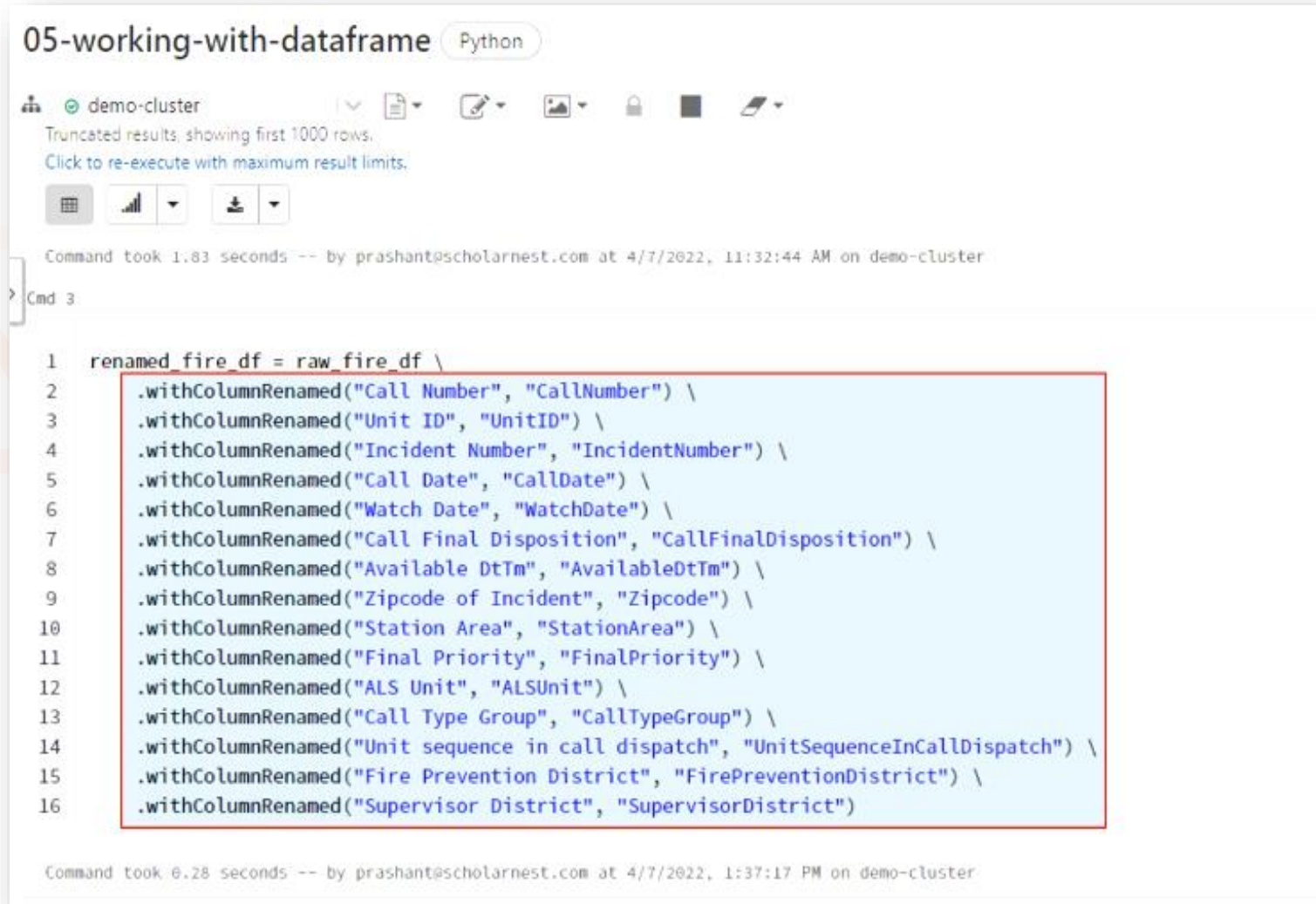
	CallNumber	Unit ID	Incident Number	CallType	Call Date	Watch Date	Call Final Disposition	Available DtTm	Addr
1	20110014	M29	2003234	Medical Incident	01/11/2002	01/10/2002	Other	01/11/2002 01:58:43 AM	10TH
2	20110015	M08	2003233	Medical Incident	01/11/2002	01/10/2002	Other	01/11/2002 02:10:17 AM	300 E
3	20110016	B02	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:47:00 AM	2000
4	20110016	B04	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:51:54 AM	2000
5	20110016	D2	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:47:00 AM	2000
6	20110016	E03	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:47:00 AM	2000
7	20110016	E38	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:51:17 AM	2000

Truncated results, showing first 1000 rows.
[Click to re-execute with maximum result limits.](#)

Command took 0.87 seconds -- by prashant@scholarnest.com at 4/7/2022, 1:33:26 PM on demo-cluster

Column Names are not standardized:

Similarly, we can rename all the problem columns as shown in the code below. If we run this code and display the results, our first problem is resolved and our column names are standardized.



The screenshot shows a Jupyter Notebook interface with a title bar '05-working-with-dataframe' and a 'Python' language selector. Below the title bar, there's a toolbar with icons for saving, running, and other notebook functions. The main area displays a code cell with 16 lines of Python code. The code uses the `withColumnRenamed` method to rename various columns in a DataFrame named `raw_fire_df` to a standardized format. The code is highlighted with a red border. Below the code cell, a status bar indicates the command took 0.28 seconds to execute.

```
1 renamed_fire_df = raw_fire_df \
2     .withColumnRenamed("Call Number", "CallNumber") \
3     .withColumnRenamed("Unit ID", "UnitID") \
4     .withColumnRenamed("Incident Number", "IncidentNumber") \
5     .withColumnRenamed("Call Date", "CallDate") \
6     .withColumnRenamed("Watch Date", "WatchDate") \
7     .withColumnRenamed("Call Final Disposition", "CallFinalDisposition") \
8     .withColumnRenamed("Available DtTm", "AvailableDtTm") \
9     .withColumnRenamed("Zipcode of Incident", "Zipcode") \
10    .withColumnRenamed("Station Area", "StationArea") \
11    .withColumnRenamed("Final Priority", "FinalPriority") \
12    .withColumnRenamed("ALS Unit", "ALSUnit") \
13    .withColumnRenamed("Call Type Group", "CallTypeGroup") \
14    .withColumnRenamed("Unit sequence in call dispatch", "UnitSequenceInCallDispatch") \
15    .withColumnRenamed("Fire Prevention District", "FirePreventionDistrict") \
16    .withColumnRenamed("Supervisor District", "SupervisorDistrict")
```

Spark Dataframe programming is all about doing two things:

1. Read your raw data into a Dataframe.
2. Transform your raw Dataframe and create a new transformed Dataframe.

We can also narrow down 3 key observations from what we have learned till now:

1. You can create a chain of Spark transformation methods one after the other.
2. Spark transformation returns a new Dataframe after transforming the old Dataframe.
3. Spark Dataframe is immutable. We cannot change or modify an existing Dataframe. Instead, we transform the existing Dataframe and create a new Dataframe.

Now let us move ahead to the second problem.

You can see call date, watch date, and AvailableDtTm fields in the data set. The call date and watch date represent a date. And the AvailableDtTm represents a timestamp. However, these fields are of the type string, which is not desirable.

05-working-with-dataframe Python

demo-cluster

Command took 0.28 seconds -- by prashant@scholarnest.com at 4/7/2022, 1:37:17 PM on demo-cluster

Cmd 4

```
1 display(raw_fire_df)
```

(1) Spark Jobs

	Call Number	Unit ID	Incident Number	CallType	Call Date	Watch Date	Call Final Disposition	Available DtTm	Add
1	20110014	M29	2003234	Medical Incident	01/11/2002	01/10/2002	Other	01/11/2002 01:58:43 AM	10TH
2	20110015	M08	2003233	Medical Incident	01/11/2002	01/10/2002	Other	01/11/2002 02:10:17 AM	300
3	20110016	B02	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:47:00 AM	200C
4	20110016	B04	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:51:54 AM	200C
5	20110016	D2	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:47:00 AM	200C
6	20110016	E03	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:47:00 AM	200C
7	20110016	E38	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:51:17 AM	200C

Truncated results, showing first 1000 rows.
[Click to re-execute with maximum result limits.](#)

Command took 0.56 seconds -- by prashant@scholarnest.com at 4/7/2022, 1:41:01 PM on demo-cluster

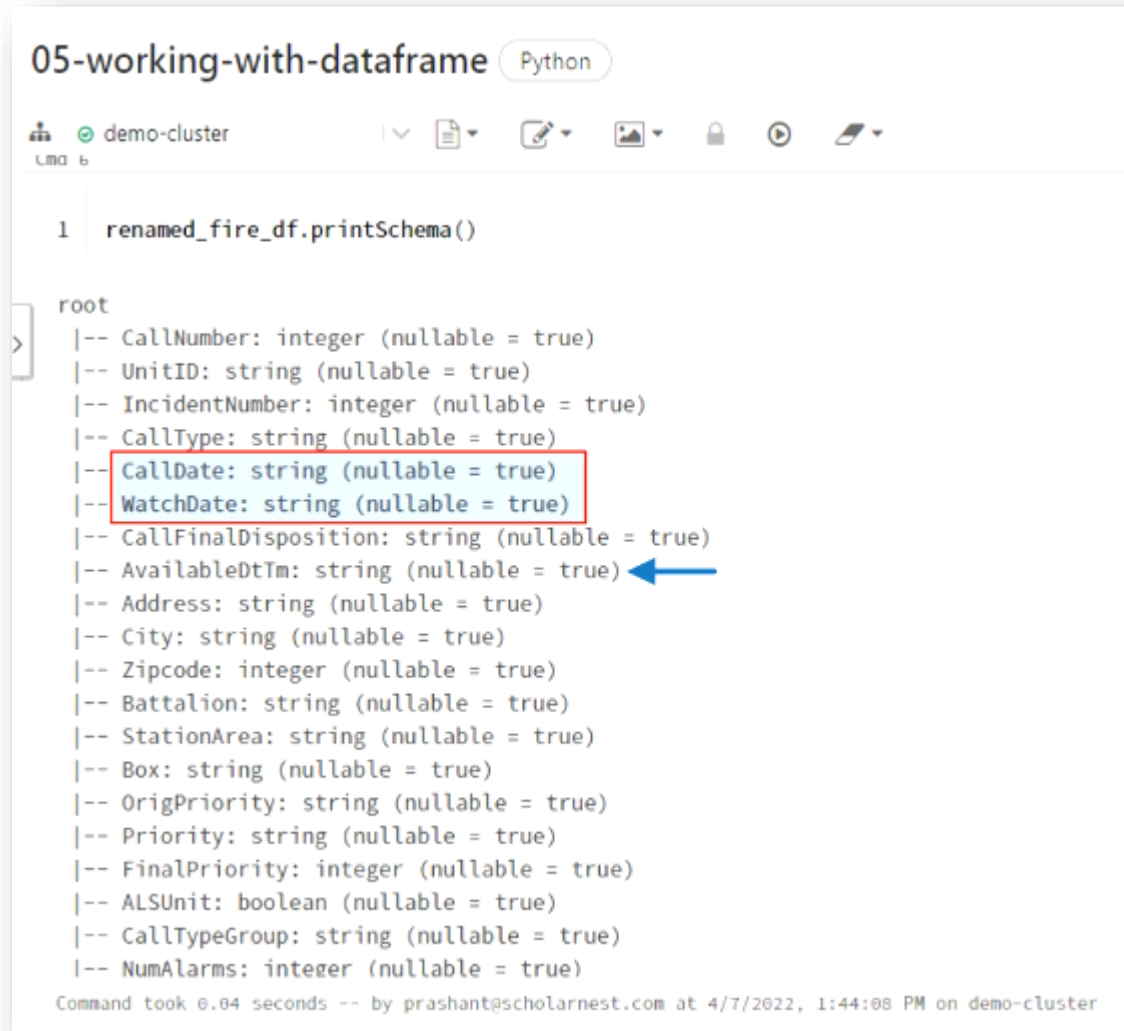
Cmd 5

1. Column Names are not standardized

2. Date fields are of string type

Date fields are of string type:

You can see the data type information using the `printSchema()` utility method. We have used this method on `renamed_fire_df` as we have transformed the older Dataframe into this new Dataframe by removing the problem column names. You can see that all the three fields are of the string type. Let me fix this problem.



```
05-working-with-dataframe Python

demo-cluster
LMA 6

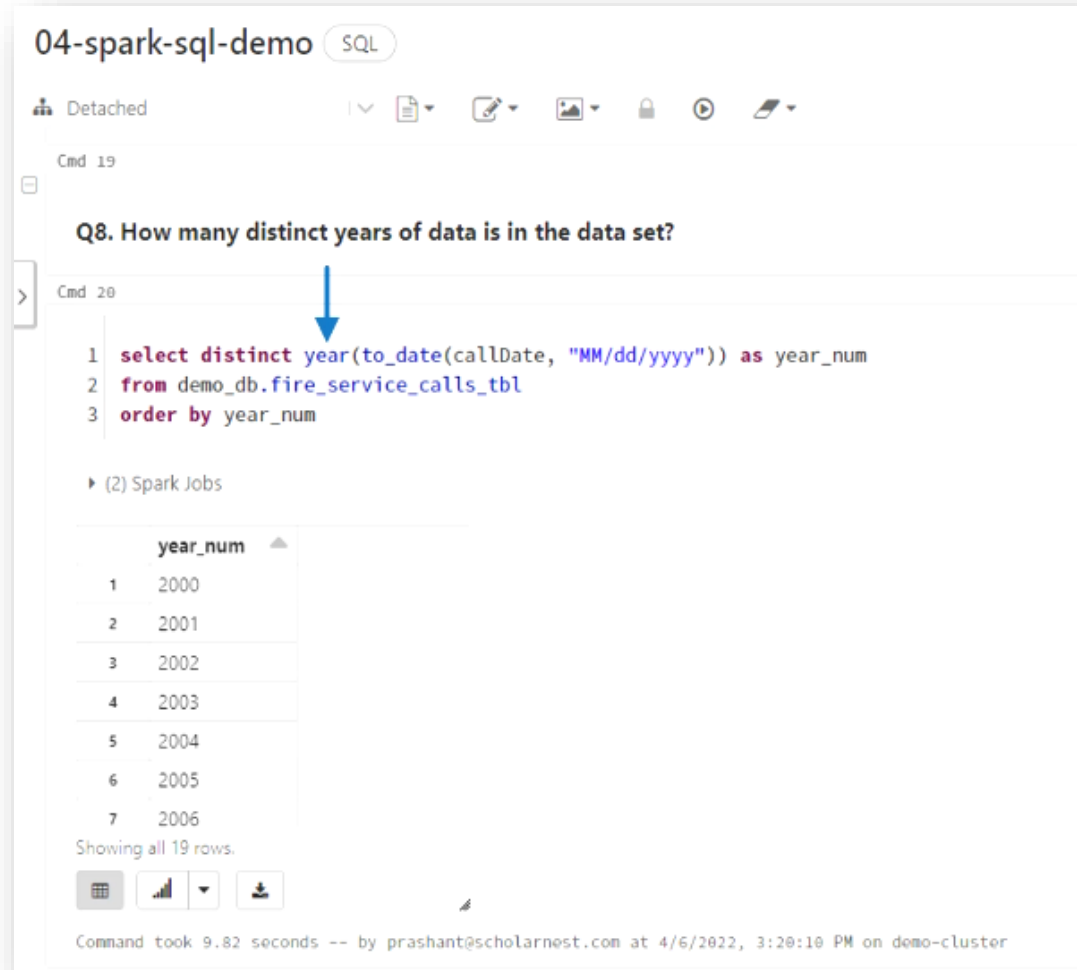
1 renamed_fire_df.printSchema()

root
|-- CallNumber: integer (nullable = true)
|-- UnitID: string (nullable = true)
|-- IncidentNumber: integer (nullable = true)
|-- CallType: string (nullable = true)
|-- CallDate: string (nullable = true)
|-- WatchDate: string (nullable = true)
|-- CallFinalDisposition: string (nullable = true)
|-- AvailableDtTm: string (nullable = true)
|-- Address: string (nullable = true)
|-- City: string (nullable = true)
|-- Zipcode: integer (nullable = true)
|-- Battalion: string (nullable = true)
|-- StationArea: string (nullable = true)
|-- Box: string (nullable = true)
|-- OrigPriority: string (nullable = true)
|-- Priority: string (nullable = true)
|-- FinalPriority: integer (nullable = true)
|-- ALSUnit: boolean (nullable = true)
|-- CallTypeGroup: string (nullable = true)
|-- NumAlarms: integer (nullable = true)

Command took 0.04 seconds -- by prashant@scholarnest.com at 4/7/2022, 1:44:08 PM on demo-cluster
```

Date fields are of string type:

We had this problem while working on spark-SQL-demo notebook. **(Reference: 04-spark-sql-demo.sql)** You can see the solution of question number 8, that we converted the Call Date to a date field. Then we applied the year() function. So, if your date and timestamp fields are stored in a string column, you may have to convert it to date on almost every use of the column.



The screenshot shows a Databricks notebook interface for a workspace named "04-spark-sql-demo". The notebook is in "SQL" mode. The command history shows two commands. Command 19 is empty. Command 20 asks "Q8. How many distinct years of data is in the data set?". Below the command, the SQL query is displayed:

```
1 select distinct year(to_date(callDate, "MM/dd/yyyy")) as year_num
2 from demo_db.fire_service_calls_tbl
3 order by year_num
```

 A blue arrow points to the `to_date` function in the query. Below the query, it says "(2) Spark Jobs". The results are shown in a table with one column, `year_num`, containing the years 2000 through 2006. At the bottom, it says "Showing all 19 rows." and "Command took 9.82 seconds -- by prashant@scholarnest.com at 4/6/2022, 3:20:10 PM on demo-cluster".

	year_num
1	2000
2	2001
3	2002
4	2003
5	2004
6	2005
7	2006

Date fields are of string type:

You can rectify this issue by transforming the *renamed_fire_df* to a new Dataframe *fire_df* by applying the *.withColumn()* transformation. The *withColumn* method takes two arguments:

1. The first column is the one you want to transform. I want to transform the Call Date column, so I give the column name here.
2. The second argument is the logic to transform the given column. We want to convert the Call Date to a Date. So I am using the *to_date()* function. And the Call Date column in the remaned_fire_df is represented using MM/dd/yyyy string. Similarly, we have the corresponding logic for AvailableDtTm.

Make sure you import the *to_date()* and *to_timestamp()* function from the *pyspark.sql.functions* package.

We are also transforming the Delay column to round the Delay column for two digits.

05-working-with-dataframe

Python

demo-cluster

```
-- ALSUnit: boolean (nullable = true)
-- CallTypeGroup: string (nullable = true)
-- NumAlarms: integer (nullable = true)
```

Command took 0.04 seconds -- by prashant@scholarnest.com at 4/7/2022, 1:44:08 PM on demo-cluster

Cmd 8

```
1 fire_df = renamed_fire_df \
2     .withColumn("CallDate", to_date("CallDate", "MM/dd/yyyy")) \
3     .withColumn("WatchDate", to_date("WatchDate", "MM/dd/yyyy")) \
4     .withColumn("AvailableDtTm", to_timestamp("AvailableDtTm", "MM/dd/yyyy hh:mm:ss a")) \
5     .withColumn("Delay", round("Delay", 2))
```

Shift+Enter to run

We can print the schema of the new fire_df Dataframe and see the data types. And we see the desired results in the output, the date and timestamp transformation is successful.

```
05-working-with-dataframe Python
demo-cluster
1 fire_df.printSchema()

root
 |-- CallNumber: integer (nullable = true)
 |-- UnitID: string (nullable = true)
 |-- IncidentNumber: integer (nullable = true)
 |-- CallType: string (nullable = true)
 |-- CallDate: date (nullable = true)
 |-- WatchDate: date (nullable = true)
 |-- CallFinalDisposition: string (nullable = true)
 |-- AvailableDtTm: timestamp (nullable = true)
 |-- Address: string (nullable = true)
 |-- City: string (nullable = true)
 |-- Zipcode: integer (nullable = true)
 |-- Battalion: string (nullable = true)
 |-- StationArea: string (nullable = true)
 |-- Box: string (nullable = true)
 |-- OrigPriority: string (nullable = true)
 |-- Priority: string (nullable = true)
 |-- FinalPriority: integer (nullable = true)
 |-- ALSUnit: boolean (nullable = true)
 |-- CallTypeGroup: string (nullable = true)
 |-- NumAlarms: integer (nullable = true)

Command took 0.03 seconds -- by prashant@scholarnest.com at 4/7/2022, 3:18:42 PM on demo-cluster
```

We learned about three new methods in this lecture.

Column Transformation Methods:

1. withColumnRenamed
2. withColumn

Utility Method:

1. printSchema()



Thank You
ScholarNest Technologies Pvt Ltd.
www.scholarnest.com